

// CONF42 AI AGENTS 2026

From Analytics to Action

Re-Architecting Master Data Management for Agentic Enterprise AI



Vishwak Thatikonda

Senior Architect, SDH Systems

ABOUT ME

Vishwak Thatikonda

Senior Architect, SDH Systems · Enterprise Master Data & Agentic AI Platforms

I architect large-scale data and platform systems for enterprises moving from analytics to autonomous action. Over roughly eleven years across retail, fintech, and mobility, I have built master data hubs, real-time resolution pipelines, and event-driven infrastructure that operate at production scale.



11+

years across enterprise platforms

38K+

retail locations on systems I have
architected

Millions

of support transactions handled

AGENDA

What We'll Cover

- 01 The shift to autonomous action, and the architecture gap it exposes

- 02 Action-grade MDM: the four pillars, real-time resolution, and the AWS stack

- 03 Operating it in production: drift, observability, and safety

- 04 A live demo, the traps to avoid, and a practical roadmap

THE SHIFT

Enterprise AI Is No Longer Just Generating Text

AI in the enterprise has moved through three phases. The third one changes what our data layer has to guarantee.

- **Observe.** Passive analytics, dashboards, and retrospective insight.
- **Recommend.** Predictive models surfacing suggestions to human operators.
- **Act.** Autonomous agents executing decisions directly against systems of record. This phase is here now: agents place inventory, manage rewards, and trigger live workflows with no human in between.

THE CORE PROBLEM

The Architecture Gap That Puts Operations at Risk

Most enterprise data architectures were designed for analytics and reporting: batch pipelines, eventual consistency, and retrospective query patterns. Those choices are all correct for a human reading a report.

In an agentic world, a stale record or an unresolved entity does not produce a misleading chart. It triggers a live operational decision.

FROM THE FIELD

When the Data Lied, the Agent Obeyed

A replenishment agent began placing duplicate orders across a cluster of stores. Nothing had shipped: no new code, no model change. Upstream, an address-normalization update had split one store into two golden records, so the agent saw two stores and ordered for both. Every order looked valid on its own, so nothing alarmed. The agent was never the problem. It executed perfectly, on data that lied to it.

9,400

Duplicate orders before anyone caught it

\$41K

Cost of a single split identity

0

Lines of agent code at fault

Why Legacy MDM Fails Autonomous Agents

● Stale Records

Batch refresh cycles leave agents operating on data that is hours or days behind operational reality.

● Near-Match Identity Errors

Fuzzy entity resolution that is acceptable for reporting creates duplicate triggers in agentic workflows.

● Duplicate Entities

Unresolved golden records cause agents to act on the same entity multiple times with conflicting payloads.

● Data Drift

Schema and value drift erodes trust in entity resolution over time, degrading accuracy silently.

The Four Pillars of Action-Grade MDM

Each pillar addresses a specific failure mode in legacy MDM when exposed to autonomous agents. Together they form a substrate agents can trust at execution time, not just at query time.

● Immutability

Append-only records with full audit lineage, so history is never silently rewritten.

● Versioned Contracts

Schema guarantees across every agent interface boundary.

● Real-Time Resolution

Sub-second match-and-survive for entity identity.

● Event-Driven Execution

State-change events that fan out to downstream agents reliably.

SCALE IN PRACTICE

Operational MDM at Enterprise Scale

Running master data across tens of thousands of retail locations demands that it serve as a live operational layer, not a reporting asset. Every decision an agent makes must be grounded in a trusted, current, unambiguous entity record.

38,000

Retail locations on one operational data layer

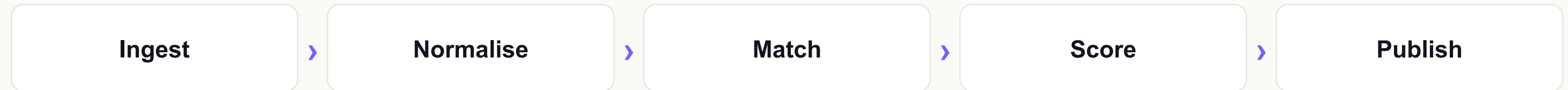
<200ms

Target end-to-end entity resolution latency

100%

Idempotent execution, zero double actions

Sub-Second Match-and-Survive



Each phase must complete within a strict latency budget to keep agentic pipelines unblocked. Target end-to-end resolution under 200ms for high-throughput entity streams.

● Deterministic Keys

Hash-based entity IDs that survive restarts and re-ingestion without collisions.

● Probabilistic Scoring

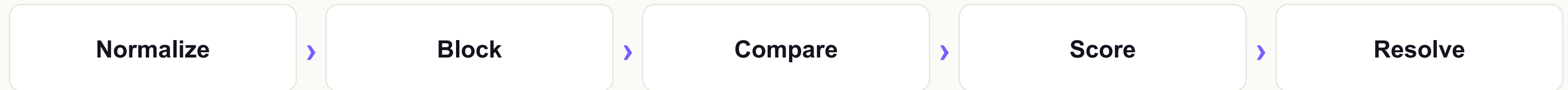
Configurable match thresholds with explicit confidence bands surfaced to agents.

● Survive Rules as Code

Source-priority rules version-controlled alongside data contracts, not buried in pipeline config.

UNDER THE HOOD

How Resolution Holds 200ms at Scale



The naive approach compares every record against every other: billions of pairwise checks. Blocking is what makes sub-200ms resolution possible, narrowing each new record to a handful of plausible candidates before any expensive comparison runs.

● Blocking

Candidate generation cuts billions of comparisons to a handful. Without it, real-time matching at scale is not possible.

● Match Signals

Name, address geometry, geo distance, and contact fields combine into one weighted, explainable score.

● Confidence Bands

Three zones, not one threshold: auto-merge, human review, and no-match. The agent is told which band it is in.

Implementing Real-Time MDM on AWS

● AWS Lambda

Stateless, event-triggered resolution functions that scale horizontally with entity volume and support idempotent re-execution on retry.

● Amazon DynamoDB

Single-digit millisecond reads for golden-record lookups. DynamoDB Streams provide the change-data-capture backbone for downstream fan-out.

● Amazon EventBridge

A schema-registered event bus for reliable, filtered routing of entity state changes to subscribing agents and downstream systems.

Idempotency Is Not Optional

In agentic pipelines the same event may be delivered more than once. An idempotent workflow produces the same outcome no matter how many times it fires, protecting against duplicate payments, inventory moves, and reward redemptions.

● Idempotency Keys

Attach deterministic request IDs at the point of event generation.

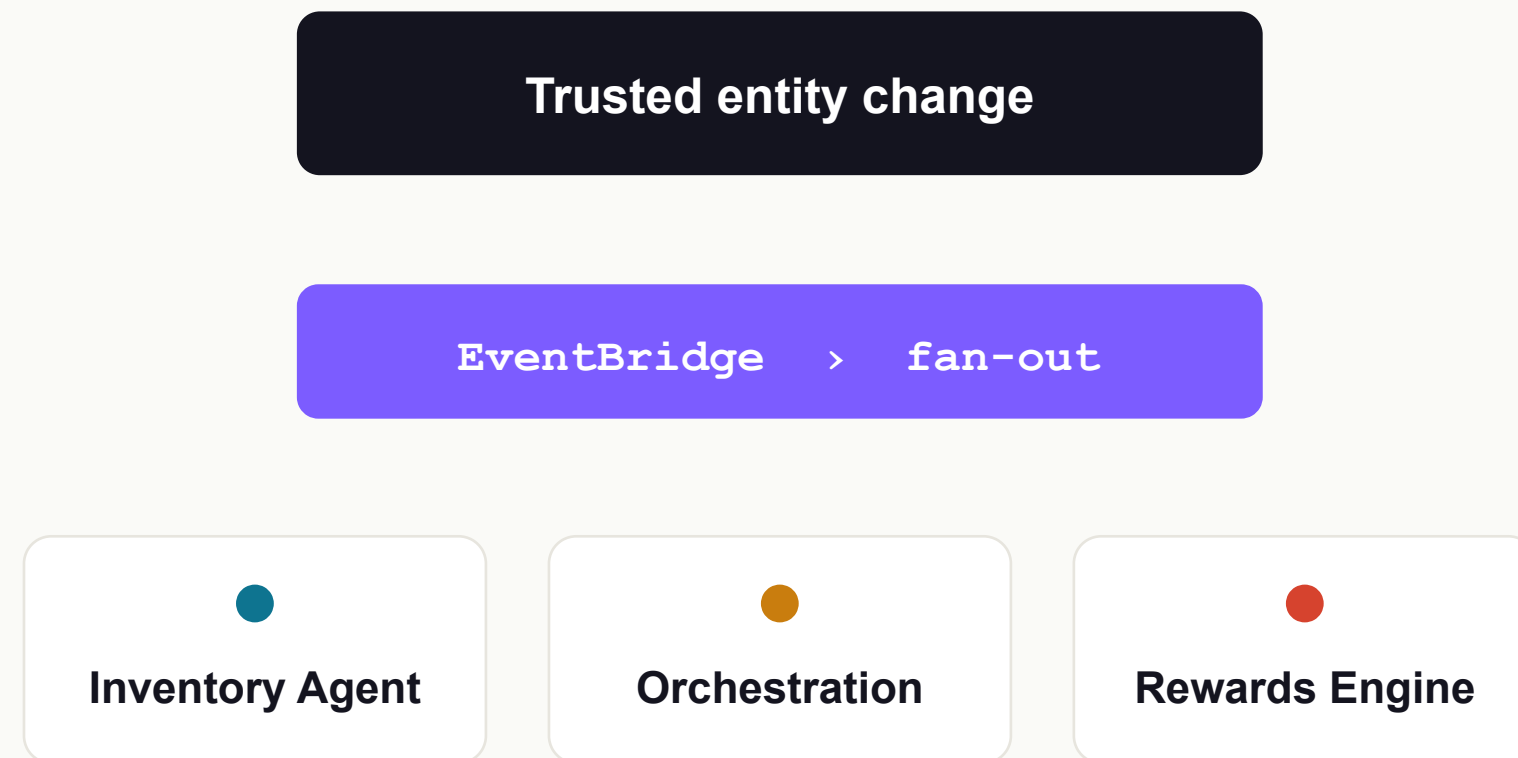
● Conditional Writes

Use DynamoDB condition expressions to reject duplicate state transitions.

● Deduplication Windows

EventBridge deduplication IDs within configurable time windows prevent re-processing at the bus level.

Event-Driven Fan-Out for Agentic Systems



One trusted entity change propagates reliably to every subscribing agent.

Why Fan-Out Matters

- Decouples agents from the MDM source: no direct polling of the golden-record store.
- Schema-registered events enforce data contracts at the routing layer.
- Dead-letter queues capture failed deliveries for replay without data loss.
- Event replay enables point-in-time reconstruction of agent decision context.

Controlling Data Drift Over Time

Drift is the silent degrader of agentic reliability. Small inconsistencies compound, eroding resolution accuracy without ever throwing an explicit error.

01

Define Drift Thresholds

Establish measurable entity-health metrics: match-confidence distributions, field completeness, and golden-record age, with automated alerts.

02

Continuous Profile Monitoring

Run lightweight statistical profiling on entity streams to detect schema drift, distribution shifts, and integrity violations in near real time.

03

Feedback Loops from Agents

Instrument agents to emit data-quality signals when resolution confidence falls below threshold, feeding corrections back into the hub.

Operating It: What You Watch

● Resolution Latency

p50 and p99 against the 200ms budget. p99 is the number that wakes you up, never the average.

● Match Quality

Precision and recall on a labeled sample, tracked continuously. A precision drop means bad merges are reaching agents.

● Contract Health

Rejections and dead-letter depth per minute. A spike is an upstream producer breaking its schema, caught before agents see it.

● Drift & Recovery

Confidence distribution, record age, idempotency rejections per hour, and replay success rate, which proves you can recover.

Versioned Data Contracts as the Agent Interface Layer

Every agent that consumes master data binds to a versioned contract defining schema, allowed values, and freshness expectations.

- Contracts are registered in a schema registry.
- Breaking changes require a new contract version.
- Violations are rejected before reaching downstream agents.

Contract Anatomy

● Schema definition

Field names, types, and constraints.

● Freshness SLA

Maximum acceptable record age.

● Ownership metadata

Producing team and steward.

When the Agent Is Wrong Anyway

The four guards stop known per-event failures. You still design for the day one slips through. These four controls bound how much damage a single bad decision can do.

● Circuit Breakers

Aggregate anomaly thresholds pause the agent when behavior spikes, even when each event looks valid.

● Human in the Loop

Actions above an impact or uncertainty threshold escalate to a person instead of executing.

● Kill Switch

One control halts an agent or workflow instantly, with state preserved for replay.

● Blast-Radius Limits

Rate caps, spend caps, and segment fences bound what any one decision can touch.

LIVE DEMO

Legacy vs Action-Grade MDM, Side by Side

One event stream hits both lanes, with an autonomous agent acting on every resolved entity. On the left, legacy MDM lets the agent do real harm. On the right, action-grade MDM blocks each failure before it executes.

● Legacy MDM

- Fuzzy miss on a duplicate store ships inventory twice
- Retried event has no idempotency: reward credited twice
- Stale out-of-order write triggers a restock on bad data
- Fuzzy merge applies one member's perk to another
- A burst of bad merges runs unchecked: blast radius unbounded

● Action-Grade MDM

- Deterministic keys and survive rules resolve identity once
- Idempotency key rejects the duplicate delivery
- Version contract rejects the stale write outright
- Confidence bands hold uncertain matches for review
- Circuit breaker trips on the burst and pauses the agent

AVOID THESE

Three Ways Teams Get This Wrong

Most failed agentic-MDM efforts do not fail on the hard engineering. They fail on three avoidable traps that feel like shortcuts and cost you later.

01

The Cache Mirage

Bolting a read cache onto a batch warehouse and calling it real-time. You get fast reads of stale truth, which is worse, because now the agent trusts it.

02

The Cleanup Project

Treating MDM as a one-time data-quality scrub. Drift makes it a living service, not a project with an end date.

03

The Over-Merge

Chasing one tidy golden record by merging aggressively. For an agent that acts, a false merge is more dangerous than an honest duplicate.

Re-Architecting MDM: A Practical Phased Approach

01

Assess

Audit existing MDM for latency, drift exposure, and identity-resolution gaps. Identify the highest-risk agent touchpoints.

02

Harden

Introduce immutable record patterns, deterministic entity keys, and idempotency controls on critical entity domains.

03

Stream

Replace batch pipelines with event-driven CDC. Deploy EventBridge fan-out with schema-registered contracts.

04

Operate

Instrument drift monitoring, agent feedback loops, and contract versioning. Establish MDM as a live operational service.

Key Takeaways for Architecture Teams

- **Assume agents will act, not advise**

Design MDM for execution-time trust, not query-time convenience. Every record must be action-ready.

- **Immutability and idempotency are non-negotiable**

In autonomous pipelines, retry and re-delivery are guaranteed. Your data layer must handle both gracefully.

- **Data contracts are your agent API**

Treat schema agreements with the same rigor as service contracts. Version them. Enforce them at the bus.

THANK YOU

Let's Keep Talking

WEBSITE

vishwak.dev

X

@whizwak

LINKEDIN

/in/vishwakthatikonda